

PeopleIntel

A lead-research dashboard. A CSV of contacts goes in; a browsable set of researched profiles comes out — who the person is, what they do, what they are currently interested in, and what their company does.

27 CONTACTS	21 COMPANIES	5823 LINES OF PYTHON	8 COMMITTS
-----------------------	------------------------	--------------------------------	----------------------

PILLAR 1 Company What the employer does, researched from their own site in preference to anyone else's.	PILLAR 2 Designation The title on file and the title on the web, kept separately — the file's copy goes stale.	PILLAR 3 Person Who they are, what they have published, and what they are actually talking about.
---	--	---

Python only. Node is not used at runtime and is not needed to run this. FastAPI, Jinja2, SQLite via SQLAlchemy, a web-search API for research and any OpenAI-compatible endpoint for the one inference step.

Generated 07 September 2026 from the working tree on branch **tanveen**. Counts in this document are read from the code and the database at build time — rebuild with `python tools/build_project_pdf.py`.

CONTENTS

What is in this document

- 1 The problem, and the shape of the answer**
Why the tool exists and what it refuses to do
- 2 How it runs**
Install, the two API keys, the CLI
- 3 The nine steps**
Upload through to reports, and the file each lives in
- 4 Architecture**
Modules, routes, pages, and what depends on what
- 5 The data model**
Six tables, and why provenance is a column
- 6 Two statuses, not one**
The bug that produced the rule
- 7 What the CSV importer tolerates**
Delimiters, encodings, a broken header row
- 8 LinkedIn, without logging in**
Three ways a post is proven to be theirs
- 9 Web research and corroboration**
Namesakes, and the three-signal test
- 10 Interests, focus lines and comment drafts**
The only LLM calls, and their guardrails
- 11 Opportunities**
Matching capability to evidence, with the field cited
- 12 Segmenting the list**
Country and category, derived on read
- 13 The outreach sequence**
Four manual steps, one open at a time
- 14 Where the data stands**
The live numbers in this build
- 15 Build history**
What shipped, and in what order
- 16 Limits and what is not built**
Stated plainly, on purpose

One rule runs through all of it

Every fact the app fetched itself stores where it came from and when. Anything that cannot be tied to the contact is not stored — not stored behind a badge, not stored greyed out. An empty panel that says “Nothing found” is a correct answer; four plausible rows about four different people is not.

That single choice is why several sections below read as lists of things deliberately given up.

SECTION 1

The problem, and the shape of the answer

A prospecting export gives you rows. It does not tell you who any of these people are. A 75-column Apollo CSV carries a name, a title that may be two years out of date, an employer, and little else you would want to open a conversation with.

PeopleIntel takes that file and produces, per contact, a page a person can read before making contact: a composed profile paragraph, what their employer does, what they have published, what they appear to be focused on right now, what can be offered to them, and a four-step outreach sequence tracking where the conversation has got to.

Three pillars

Company, Designation, Person. Every panel on the dashboard answers one of those three, and the data model is arranged around them: Company holds the employer and its researched description, Person holds identity and both copies of the designation, and the four child tables hold what was found about them.

What it deliberately does not do

- **It never logs into LinkedIn.** No credentials, no scraper, no third-party LinkedIn data provider, and it never fetches a linkedin.com page. Public post URLs reach the panel through the same web-search index used for everything else, or by hand.
- **It never guesses an email address.** No first.last@domain pattern construction. A pattern produces a plausible string, not a fact, and a plausible string in an email field is worse than an empty one because nothing downstream can tell them apart.
- **It never infers an interest from a job title.** Thin evidence returns an empty panel. Every interest chip carries the evidence that produced it, and a chip with an empty evidence field is dropped before it is displayed.
- **It never sends anything.** No comment is posted, no connection request is sent, no email goes out. The outreach sequence tracks what a person did, and drafts a comment for them to edit — automated LinkedIn engagement is exactly the thing a hand-written first touch is trying not to look like.
- **It never silently overwrites the file.** Where research disagrees with the CSV — a job title, a LinkedIn profile — both values are kept and the mismatch is flagged.

SECTION 2

How it runs

Install and start

```
pip install -r requirements.txt

python -c "from app.db import init_db; init_db()"
uvicorn app.main:app --reload --port 8000
```

Open `http://127.0.0.1:8000`, upload a CSV, and the contacts appear. Six runtime dependencies, all Python: FastAPI, uvicorn, Jinja2, python-multipart, SQLAlchemy, httpx. `static/app.css` is committed prebuilt, so there is no build step and no Node — Tailwind is only needed to change styles, and its standalone binary needs no Node either.

Loading the sample data

```
python -c "
from app.db import init_db, SessionLocal
from app.importer import import_csv
init_db(); db = SessionLocal()
print(import_csv(db, open('data/apollo_contacts.csv', 'rb').read()))"
python seed_research.py
```

The two API keys

Variable	What it powers	Without it
FIRECRAWL_API_KEY	Web research: company descriptions, the Google/Web panel, the LinkedIn activity link search, and every enrichment lookup in resolve.py.	run_pipeline.py exits immediately. The app still runs and still shows everything the CSV gave it.
LLM_BASE_URL LLM_MODEL LLM_API_KEY	Interest chips, the focus line, and the recommended comment drafts. Any OpenAI-compatible chat-completions endpoint — Groq now, a self-hosted Ollama or vLLM by changing the base URL and nothing else.	Research still runs; the interest and draft steps are skipped. --skip-interests does this on purpose.

Read at startup from .env by app/env.py; a variable already set in the shell wins over the file. The server does not watch the file, so a change needs a restart — and /settings shows which keys are present without ever printing their values.

The pipeline CLI

Command	What it does
python run_pipeline.py --limit 5	Research five outstanding contacts. The sensible first run — it costs credits, so try a few before all of them.
python run_pipeline.py	Everything still outstanding.
python run_pipeline.py --force	Re-research contacts that already have results. Deliberate, and the reason the UI's Research button is not offered for a contact that already has results.
python run_pipeline.py --skip-interests	Research only. No LLM calls at all.
python run_pipeline.py --skip-linkedin	Skip the activity-link search.
python run_pipeline.py --only-linkedin	Re-run just the activity search for every contact, without paying again for company and web research that already succeeded.

SECTION 3

The nine steps

The pipeline the whole tool is organised around. Steps 4 to 7 are the ones that reach the network; the rest are local.

#	Step	Where it lives, and how it works
1	CSV upload	app/importer.py — alias-matched columns, sniffed delimiter, preamble skipped, unusable header rows repaired. Every tolerance is reported on the upload page rather than applied silently.
2	Parsing and normalization	75 Apollo columns down to the ~25 fields the UI actually uses. Rows are read positionally, so a duplicated column name can never swallow another column's data.

#	Step	Where it lives, and how it works
3	Missing-data enrichment	Gap detection per contact via <code>Person.missing_fields()</code> ; the lookups themselves in <code>app/resolve.py</code> , with <code>app/employer.py</code> working out who someone works for when the CSV did not say.
4	LinkedIn activity	<code>app/research.py</code> — public post links from the search index, plus hand-pasted entries. LinkedIn is never crawled. Section 8.
5	Google / web research	<code>app/research.py</code> — two query angles, each result kept only if it can be tied to the contact. Section 9.
6	Interest detection	<code>app/interests.py</code> — one OpenAI-compatible call producing the chips and the focus line, both required to cite their evidence.
7	Company research	<code>app/research.py</code> — the company's own site preferred over anyone writing about them.
8	Dashboard	FastAPI plus server-rendered Jinja2. Interactivity is plain forms — no client framework, no build step.
9	Reports and export	<code>/reports</code> — breakdowns, and the stale-designation list.

One definition of “research a contact”

The CLI and the Research button in the UI both call `app/pipeline.py:research_contact`. Two copies of that sequence would drift, and the one used less often would be the one that rotted. The same reasoning extracted `refresh_linkedin` when the standalone LinkedIn refresh was added.

The button posts to `/person/<slug>/research`, parks the contact at *Researching...* and returns immediately — a run takes tens of seconds, and longer when the API rate-limits and the backoff waits, so the work happens in a thread. The form's back field is restricted to in-app paths, so the endpoint cannot be turned into an open redirect.

SECTION 4

Architecture

Server-rendered pages over a SQLite file, with two outbound HTTP dependencies. 5823 lines of Python across 16 modules, and 2219 lines of Jinja2 templates.

The stack, and why

Layer	Choice	Note
Web framework	FastAPI	Dependency-injected sessions; the same app object serves HTML and the form posts.
Templates	Jinja2, server-rendered	Panels are macros in <code>templates/_partials.html</code> , so the dashboard and the person page render the identical component.
Interactivity	Plain HTML forms	One JS timer, polling a running LinkedIn refresh. Nothing else.
Styling	Tailwind, prebuilt	<code>static/app.css</code> is committed. No Node at runtime.
Database	SQLite via SQLAlchemy	Fine on a host with a persistent disk (Railway, Render, Fly). Not fine serverless — on Vercel the filesystem is ephemeral and writes vanish between requests. Point <code>app/db.py</code> at Postgres and nothing else changes.
Web research	Firecrawl search API over HTTP	One <code>_search</code> function with backoff; every caller goes through it.

Layer	Choice	Note
Language model	Any OpenAI-compatible endpoint	No provider SDK, so hosted-to-self-hosted is a change to one environment variable.

Modules

Module	Responsibility	Lines
app/main.py	FastAPI app, every route, and the Jinja filters (relative time, short date, money, domain-of-URL).	549
app/models.py	The six tables, the status constants, and the derived properties the templates read.	709
app/db.py	Engine, session factory, and a lightweight add-missing-columns migration on startup.	90
app/env.py	Reads .env without a dependency; reports which keys are present.	68
app/importer.py	CSV reading, header repair, alias column mapping, slug allocation, and the import report.	528
app/research.py	Search, classification, corroboration, activity attribution and dating, employer naming, company research.	903
app/resolve.py	Step 3 enrichment: find a missing email or LinkedIn URL, and refuse to when the evidence is ambiguous.	375
app/employer.py	Works out the employer from title and posts when the CSV left it blank. Writes nothing.	250
app/pipeline.py	The one definition of researching a contact, and of refreshing just the LinkedIn panel.	284
app/interests.py	The chips and the focus line.	148
app/messages.py	Recommended comment drafts, and the substance check that decides whether a draft is possible at all.	269
app/summary.py	The composed profile paragraph, plus the markup cleaners the panels use on scraped text.	305
app/opportunities.py	Capability matching, tiered by evidence, citing the field that fired.	239
app/segments.py	Country and category, derived on read.	194
app/outreach.py	The four-step sequence, its waits, and what blocks a step.	247

Scripts at the repository root

Script	What it is for
run_pipeline.py	The research CLI — see section 2.
seed_research.py	Loads the researched sample state used in the screenshots, so the dashboard can be seen full without spending API credits.
backfill_activity_dates.py	One-off: decodes the timestamp out of each activity URL's id and fills activity_date for rows stored before dating existed.
purge_linkedin_web_findings.py	One-off: removes the LinkedIn rows that the web panel had collected before corroboration was tightened.
tools/build_project_pdf.py	Builds this document.

The two one-off scripts are kept rather than deleted because each took a backup first — the `data/peopleintel.backup-before-*.db` files are those backups, one per behaviour change that rewrote stored rows.

Pages and routes

Page	Route	What is on it
Home	GET /	A selected contact rendered in full — profile summary, interests, company, opportunities, LinkedIn activity and web findings — above the people list and the upload panel.
People	GET /people	The list, filterable by status, country and category, with the Research button on any contact that has not been researched.
Person	GET /person/{slug}	Everything known about one contact, the paste form for an activity, the drift warnings, and their outreach steps.
Companies	GET /companies, /company/{id}	Employers, their researched description with its source and fetch date, and their people.
Enrichment	GET /enrichment	Who is missing what, critical gaps separated from optional ones.
Research	GET /research	Research status across the list.
Reports	GET /reports	Breakdowns, and the stale-designation list.
Outreach	GET /outreach	The board: what is due today, what is overdue, what is waiting.
Settings	GET /settings	Which environment keys are present. Never their values.
Upload	GET /upload, POST /upload	The CSV form, and the import report.

Form endpoints

- POST /person/{slug}/research — research one contact, in a thread.
- POST /person/{slug}/refresh-linkedin — re-run just the activity search.
- POST /person/{slug}/suggest-comments — draft comments for their posts.
- POST /person/{slug}/activity and POST /activity/{id}/delete — paste an activity by hand, or remove one.
- POST /person/{slug}/outreach/start, and /outreach/{step_id}/{done, skip, reschedule} — move a contact through the sequence.

SECTION 5

The data model

Six tables. The design rule, carried from the first plan: every fact that was fetched from the web stores where it came from and when. A field that came straight from the CSV carries no provenance, because the file is the source.

Table	Rows	What it holds
companies	21	The employer as the CSV gave it — name, website, industry, headcount, location, revenue, funding, technologies, keywords — plus the researched description with its source URL and fetch time.
people	27	Identity, contact details, location, both statuses, and the observed-versus-filed pairs for designation and LinkedIn profile.

Table	Rows	What it holds
linkedin_activities	47	One post, repost, comment or tagged mention. Carries its type, URL, text, date, display rank, whether it was pasted or found by search, the query that found it, its corroboration, and the recommended comment draft.
web_findings	35	One search result: title, URL, snippet, kind, rank, the query, whether it was corroborated and by what, and whether it is company-news filler rather than about the person.
interests	37	One chip: its label, the evidence that produced it, and whether that evidence was LinkedIn, the web, or both.
outreach_steps	9	One step of the sequence for one contact: due date, done or skipped and when, the skip reason, and a note recording what was actually done.

Both copies, never one

Two fields on Person exist in pairs because the CSV's copy goes stale and the app's answer is not automatically better:

From the file	Found on the web	Flagged as
title	title_observed, with its source and the date it was seen	title_drift — compared loosely, so “Director, Content” and “director content” are the same title
linkedin_url	linkedin_observed, the profile they actually post from	linkedin_drift — compared on the slug, so http/https, www and a trailing slash do not differ

Neither overwrites the file. The person page says so where they disagree, and /reports lists every designation mismatch in one place.

Derived on read, never stored

A property that can be computed from the record is a property, not a column — so a re-upload or an enrichment run cannot leave a stale copy behind. profile_summary, opportunities, category, segment_countries, outreach_stage, title_drift and linkedin_drift are all computed when the page asks for them.

One legacy column is kept deliberately: Person.status, which nothing in the app reads any more, is held in step with enrichment_status so that an older query or export does not silently read a frozen value.

SECTION 6

Two statuses, not one

The bug that produced the rule

One field answered two questions, so “Needs Enrichment” came to mean “something, somewhere, is missing” — which is true of almost every contact and therefore says nothing about any of them. It fired when research had not run yet, or when a company had no description, and flagged 16 of 25 contacts.

A fully identified person who has not been researched is not missing data about themselves. A person with no email is, whether or not research has run.

Field	Question it answers	Values
enrichment_status	Do we know who this person is? Set from identity fields only: email, LinkedIn URL, designation, and the employer's website. Each is a thing enrichment could go and fill. Nothing else counts.	Complete · Needs Enrichment
research_status	Has anyone looked them up? A failed search no longer overwrites the enrichment verdict — the two are independent.	Not researched → Researching... → Researched, or Research failed

Because they are independent, **Needs Enrichment + Researched** is a real, expressible state, and so is **Complete + Not researched**. Both are visible in the live data (section 14).

Explicitly not enrichment blockers

Gap	Still shown as	Why it does not block
Direct phone	an optional gap	The Apollo export carries one for nobody, so counting it would hold every contact at Needs Enrichment forever.
Company description	an optional gap	A fact about the employer, and one that research fills in — not something missing from this person's record.
Web research, LinkedIn activity	an optional gap	Their absence means nobody has looked yet, or there was nothing public to find. That is research status.

Both statuses are decided in exactly one place, `Person.recompute_status()`. Five call sites previously each ran their own copy of the rule, so a change to it had to be made five times. The method also guards a run in progress: without that guard an import or an activity edit mid-run would reset the badge to “Not researched” and the button would offer to start a second run.

Designation counts as critical because it is one of the three pillars and it comes from the file, so a blank one is a real hole. Dropping it from `missing_critical()` is a one-line change and nothing else has to move.

Two buttons, two different rules

	Research	Refresh LinkedIn
Offered when	Nothing found yet, or the last attempt failed.	The first research run has completed at least once, and no run of either kind is in flight.
Offered again after success?	No. Re-running spends API credits, so that is a deliberate <code>run_pipeline.py --force</code> , not something a stray double-click can do.	Yes, every time. Activity is the one genuinely time-sensitive panel here — LinkedIn's feed moves daily, so a search-found list from weeks ago goes stale in a way the rest of the record does not.
Progress shown by	<code>research_status = Researching...</code> , the whole record.	<code>linkedin_refreshing</code> , a separate flag, so refreshing one panel does not flip the whole record back to “Researching...”. A JS timer polls it.

SECTION 7

What the CSV importer tolerates

The 75-column Apollo export is the happy path, but the file that arrives is often not that. Each tolerance below is reported on the upload page rather than applied silently.

Case	Handled how
Delimiters	Comma, semicolon (Excel under a European locale), tab, pipe. Sniffed from the file's content, not its extension.
Encodings	UTF-8, UTF-8 with BOM, UTF-16 (Excel's "Unicode Text"), Latin-1.
A preamble	Lines above the header row — <i>Exported from CRM on ...</i> — are skipped by finding the real header.
Other exports	LinkedIn Sales Navigator, HubSpot, Salesforce, and a plain Name, Email file all import. Columns are matched by alias, so <i>Associated Company</i> , <i>Account Name</i> and <i>Company</i> all mean the same field.
A broken header row	An export sometimes arrives with its leading block of column names overwritten by one repeated label — 17 columns all called <i>First Name</i> . A repeated label names none of its columns, so those positions are recovered from the Apollo layout by aligning on the labels that <i>are</i> trustworthy. Below 60% agreement the repair is refused rather than guessed at, and where a label still repeats, the first column carrying it is the one that counts.

Rows are read positionally

Never by column name. A duplicated column name can therefore never silently swallow another column's data — which is the failure mode the header repair above exists to survive in the first place.

Then 75 columns become 25

Step 2 keeps only the fields the UI uses, splitting them across the employer and the person and coercing the numeric ones (headcount, revenue, funding). Each new contact also gets a unique slug, allocated against the slugs already in the database so that two people with the same name get distinct, stable URLs.

SECTION 8

LinkedIn, without logging in

No credentials, no scraper, no data provider

This app has no LinkedIn credentials, no scraper, and no third-party LinkedIn data provider. It never logs into LinkedIn and never fetches a linkedin.com page. Activities reach the panel two ways, and every row says which.

Route	How the row arrives	Rows now
found by search	The pipeline and the Research button ask the same web-search index used for everything else for public post URLs — <code>site:linkedin.com/posts "Name"</code> . Each row keeps the query and the fetch time, the same provenance a web finding carries.	44
pasted	The form on the contact's page: pick the type, paste the link and the text. A hand-pasted row is trusted outright — a person judged it — and is never overwritten by a later search run; the search only fills the slots left over, up to five in total.	3

What proves a post is theirs

A post URL is `/posts/<author-slug>_<words-from-the-post>-activity-<id>`, so the author segment is the only thing in a search result that can prove authorship. A row is kept only when one of three things ties it to *this* contact:

1. **The author slug is a profile we hold for them**, compared exactly. Not as a substring: the short slug `anwar-chaudhry` is itself a substring of `dr-mumtaz-anwar-chaudhry-98231b11`, and a post's own text-slug contains the names of everyone it talks about — so substring matching credited a hospital's post *about* a Professor Anwar Chaudhry to ours.
2. **The author is a different profile carrying their name, and the post names their employer**. Two independent facts — the same pair `resolve.py` accepts as proof that a LinkedIn URL belongs to someone.
3. **Someone else's post that names them with their job title**, where the author is the employer's own page, or where name, employer and title all three appear. That is the three-fact test the web panel already uses.

Name alone is never enough.

Matching on name tokens credited five different Laura MacLeods' posts to ours. A middle version that showed name-matched rows behind an “unverified” badge was built and then reverted: of 49 rows across 15 contacts, 7 were the actual contact. One panel showed four different Christopher Carrolls; another showed a recruiter called Daryl Daley, matched because “daryl” and “speed” appear in that order in “Great insight, Daryl! speed with clarity”. A labelled wrong row is still a wrong row on someone's profile.

Why rule 2 exists

Requiring the CSV's slug meant the panel was empty whenever the export's LinkedIn URL was not the profile the person posts from — and vanity URLs get changed, so exports go stale. Julie LeBrun posts weekly about OCA training from `julie-lebrun-tumbaoju` while the file records `julie-lebrun-45583110b`; her panel showed nothing, which reads as “she does not post” rather than “our URL for her is wrong”.

When rule 2 fires, the profile it found is stored in `linkedin_observed` and the contact's page says the file's URL points elsewhere — because the header link and the outreach board's “Open their LinkedIn” both go to the wrong profile until someone fixes it. A contact with no `linkedin_url` at all is not excluded; it just narrows them to rules 2 and 3.

Ordering the panel

Kind first, then recency inside a kind: the contact's own readable posts, then anything else they wrote, then anything readable, then the rest — newest first throughout, with the date decoded from the activity id embedded in each URL.

Recency alone was right while every row was a post they wrote. Once posts that merely name them are admitted it is not: those arrive in volume and are mostly LinkedIn's attribution block with a fresh date on it, so recency alone ranked a content-free February mention above Gordon Hirons' own announcement of the IB Science Questionbanks.

Snippets that say nothing

When a contact comments under someone else's post, what the index returns is often just LinkedIn's furniture — “*Micaela Metz, graphic · Micaela Metz. Senior Learning Content Manager @ Axonify. 1y. Report this comment*”. Strip the chrome, their name, their title and their employer, and nothing is left.

Those rows keep their link, type and date and lose their text, because text is what interest chips are derived from — and a chip derived from that snippet would be a chip derived from a job title.

Two honest limits on the searched route

It sees only what a search engine has already indexed, which for a mid-level contact is often nothing. The panel says **“Nothing indexed”** rather than padding to reach five.

The text is the indexed snippet, not the whole post. Where no snippet exists the row is kept as a link and labelled as one — and that label matters, because a bare URL gives the interest model nothing to work with. Pasting the real text by hand is still worth doing.

SECTION 9

Web research and corroboration

A search for a name returns namesakes. Searching *“Laura Macleod” Nelson* returned her real LinkedIn profile — and a Maine phone listing, a freelance creative strategist, a wedding-group singer, and a 1994 SAGE paper on teaching performance appraisals. Four different people.

So a result is kept only when one of three things is true:

1. It is on the employer's own domain, or a subdomain of it.
2. It is the contact's own LinkedIn profile, matched on the slug from the CSV.
3. Its text carries their name, their job title **and** their employer, together.

Signal 3 exists because 1 and 2 alone were too narrow: *“Nora Mawla. Content Marketing Manager, Corndel.”* is unmistakably her, and was being discarded for the sole reason that it was not hosted on corndel.com. Three CSV facts co-occurring is not a coincidence a namesake produces. A name alone is not a signal, and neither is a name plus an employer — “Nelson” matched a post written by a Sean Nelson.

Anything that clears none of the three is not stored

An earlier version kept them behind an “identity unconfirmed” badge, on the theory that a labelled lead beats an empty panel. It does not: the label puts the work of re-checking every row back on the reader, and a stranger's phone listing sitting on someone's profile is wrong whatever it is wearing.

The cost is real and was measured. Of 85 findings re-scored under this rule, 18 were recovered by signal 3 and 36 were dropped — and a few of those were genuinely relevant, like a conference-talk listing that simply never repeats the speaker's title and employer in its indexed snippet. Precision was chosen over recall deliberately. Loosen signal 3 in `research._corroborate` to trade back the other way.

Coverage is uneven, and that is the data

A senior, conference-speaking contact returns four or five solid findings. A mid-level Content Manager often returns only their own LinkedIn profile. Of the 25 contacts in the sample export, 13 are Manager-level — thin panels are expected for most of them. Empty panels say “Not found” and stay empty.

One narrow exception exists: where the person searches turned up too little, company news can fill the panel, stored with `about_company = true` and never corroborated. By construction it can never derive an interest chip or an inferred employer — both already skip anything uncorroborated — it only keeps the panel from sitting empty, and it says on its face that it is about the employer.

Designations go stale fast

Of the four contacts researched early in the sample, **three** had a different job title on the web than in the file:

Person	In the CSV	On the web
Laura Macleod	Executive Director, Content Services	Vice President , Content Services
Joshua Dyer	Director, Content & Community Engagement	Director, Strategy & Engagement
Angelina Attisano	Content Manager	Communications & Administrative Professional

Designation is one of the three pillars and it ages faster than anything else in the row. Mismatches are flagged on the contact's page and listed on /reports. Both values are kept — the tool neither silently overwrites the file nor silently trusts it.

SECTION 10

Interests, focus lines and comment drafts

These are the only inference steps in the project, and they are written against the OpenAI-compatible chat-completions format — no provider SDK — so the same code runs on Groq now and a self-hosted Ollama or vLLM later by changing LLM_BASE_URL.

Interest chips and the focus line

The panel is the most trust-sensitive part of the page, so the prompt enforces two rules and the code enforces them again:

- **Every chip must cite the evidence it came from.** A chip with no evidence is a guess dressed as a fact, and one arriving with an empty evidence field is dropped in `app/interests.py` rather than displayed. So no chip on the page is unsourced — hover one to see the post or article that produced it.
- **Thin evidence returns nothing.** MIN_EVIDENCE is the floor: fewer signals than that and the step does not guess. An honest empty panel beats four plausible chips invented from a job title.
- Only corroborated rows are eligible as evidence in the first place, and what the person themselves said is preferred over what others wrote about them.
- Chips are 1–3 word noun phrases, at most five. The focus line is one or two sentences about what they are actually working on, or `null` when the evidence is too thin.

The composed summary, for contrast

The profile paragraph on every contact is **composed, not generated**. Every clause restates a field the app already holds — title, seniority, department, company, industry, headcount — so it exists for every contact the moment a CSV lands, costs nothing, and cannot invent. That matters more here than fluency: those fields are present for everyone, while the focus line is present for 11 of 27. A generated summary would have left most of the dashboard blank.

Research is folded in when it exists and skipped silently when it does not, so the paragraph gets richer as the pipeline runs rather than appearing late. `basis` names the fields behind each sentence, so the panel shows its sources the way every other panel does. And **no pronoun is ever guessed** — a name does not tell you how someone refers to themselves, so the subject is the person's name and then “they”.

Recommended comment drafts

Steps one and two of the outreach sequence are “comment on their post”, and a task that says only that leaves the actual work undone. So the draft lives beside the post it replies to, for a person to edit and send. Nothing is posted automatically.

The hard constraint is the source text.

An activity row's text is the search index's snippet of a LinkedIn page, and for most rows that snippet is page furniture rather than the post: “*Nora Mawla Content Marketing Manager Videographer Content Creator 8mo*”. Of the 17 rows on hand when this was built, 7 carried something a comment could reference and 10 carried the person's

own headline and nothing else.

So substance is checked before anything is generated. A draft written from a headline would be a confident comment about nothing — the exact failure the rest of the app refuses elsewhere — and it would go out under the user's name. Rows that fail the check say so, and point at the paste form, which is the actual fix: pasting the real post text makes a good draft possible.

The output is checked in code, not merely requested in the prompt

Flattery openers, pitches, links and hashtags are rejected outright, because a comment carrying any of them reads as automated — and that defeats the entire point of a hand-written sequence. `suggested_note` carries the reason when no draft was possible, so the panel explains itself rather than sitting empty.

SECTION 11

Opportunities

What can be offered to the contact on screen. Screwdriver's four capabilities are matched against signals already on the record. Nothing is generated and nothing is guessed — a capability appears only when a term for it is actually present in the person's own words, their designation, or their employer's record, and the row names the exact field it fired on and quotes the words around the match.

Capability	What is offered	Example terms it fires on
AI & Automation	AI-assisted content pipelines — auto-tagging, transcription and repurposing of the library they already own.	ai, machine learning, automation, agentic, llm, generative, copilot
Video & Multimedia	Production capacity, scripted, edited and versioned for their channels, without standing up an in-house studio.	video, videography, multimedia, podcast, film, animation, webinar
Custom Software	The portals, internal tools and integrations that sit between their content and the people meant to use it.	platform, software, engineering, integration, lms, portal, api, saas
Learning Design	Instructional design and production for their programmes, from outline to published module.	learning, e-learning, curriculum, certification, instructional, pedagogy, upskilling

Three tiers, because the evidence is not equally about the person

Tier	Source	What the row says, and its caveat
said	Their own posts, interests or focus line.	<i>"They raised it themselves"</i> — the only tier that reflects this person's stated priorities.
role	Their designation and department, from the CSV.	<i>"It sits in their remit"</i> — solid about what they own, but a job description is not an intent.
company	The employer's industry, description and keywords.	<i>"Context at their employer"</i> — weakest: true of the employer, not necessarily of the person reading the pitch.

The tier is shown on every row, so a reader can tell "they said this" from "their employer is in this sector" without opening anything. An unmatched capability is left out rather than padded in — a plausible-looking pitch nobody can trace is worse than a shorter list.

Matching is per field, never against a merged blob

An earlier version concatenated a contact's industry, description and CSV keywords and searched that. It found the right term but then cited the wrong source — “*Hibernia College works in this space — e-learning*” for a term that had in fact matched in the keywords column. A citation that points at the wrong field is worse than no citation, because it looks checkable and is not.

Two smaller details follow from the same care: terms match on word boundaries, so *ai* must be the word “ai” rather than firing on *said*, *email* and *training*; and the quoted excerpt is centred on the match, because truncating from the start would routinely cut off the very word that justified the row.

SECTION 12

Segmenting the list

Two independent filters. Either one alone narrows the list, and together they intersect — “Education in Canada”. Neither is stored on the record; both are derived on read from what the CSV already gave us, so a re-upload or an enrichment run never leaves a stale segment behind.

Country — either country matches

An Apollo export carries two countries, and they are not the same fact: where the person is, and where their employer is. On the list in hand the person's country is Canada for 25 of 27 rows while the employer's spans seven countries, so filtering on the person alone would produce one useless bucket. A contact therefore matches a country when *either* field matches, and both countries are shown on the row so a match is never mysterious.

Employer countries in the current data: Canada (10), United States (5), United Kingdom (2), Switzerland (1), Ireland (1), Austria (1), Australia (1).

Category — keyword-mapped, first match wins

Nothing in the file says “education” or “medical”. The nearest thing is the employer's industry, which arrives as free text — *e-learning*, *higher education*, *pharmaceuticals*, *electrical/electronic manufacturing* — so those are mapped into eight broad buckets by keyword.

Bucket	Matched on terms like
Education	e-learning, edtech, university, curriculum, training, courseware, pedagogy
Medical & Health	medical, hospital, pharmaceutical, biotech, clinical, nursing, diagnostic
Engineering & Technology	engineering, software, semiconductor, manufacturing, robotics, saas, cloud, cyber
Finance & Insurance	financial, bank, insurance, investment, accounting, fintech, wealth
Nonprofit & Public	nonprofit, charity, civic, government, public administration, ngo, foundation
Media & Creative	media, publishing, film, broadcast, advertising, design, journalism
Retail & Consumer	retail, consumer, apparel, food, hospitality, tourism, e-commerce
Other	Matched nothing above.

Order is precedence, so *professional training & coaching* has to reach Education before anything else claims it. When the industry is blank, the company's keywords and then the person's own title are tried — a row with no industry still often says what it does, and a title of “*Filmmaker. Creative Director. Founder*” is clearly Media. A row that matches nothing lands in **Other** rather than being hidden: an unmatched contact is still a contact, and silently dropping it from every category view would quietly shrink the list.

SECTION 13

The outreach sequence

Comment, wait, comment, connect, email. Every step is done by hand: nothing here posts a comment, sends a connection request or sends an email. The app tracks where each contact is and what is due today, and a person does the work.

Why it is manual on purpose

Automated LinkedIn engagement gets accounts restricted — and an automated first touch is exactly the thing this sequence is designed not to look like. The one piece of help the tool offers is a comment draft (section 10), which a person still edits and sends.

Step	Wait	Counted from	What it is
1 • Comment on their post	0 days	entering the sequence	Pick a recent post from the LinkedIn Activity panel and leave something specific. The panel is ordered newest first.
2 • Comment again	2 days	step 1 completed	The two-day gap, so the second comment is not same-day.
3 • Connect	0 days	step 2 completed	Straight after the second comment, while they may still recognise the name from the comment thread.
4 • Email	2 days	step 3 completed	Two days after the connection request.

Those are defaults, not rules. A due date can be moved, and a step can be skipped with a reason when there is nothing to comment on.

Why steps are created one at a time

A manual sequence cannot schedule step three in advance, because its due date depends on the day step two *actually* happened rather than the day it was supposed to. So a step is created when the one before it is completed, with its due date measured from that completion — marking a step done is therefore also what schedules the next one.

Exactly one step per contact is open at a time, which is what makes “today’s tasks” a simple query rather than a scheduling problem. A step is open until it is either done or skipped, and reports one of five states: **waiting**, **due**, **overdue**, **done**, **skipped**. Each step also carries a note recording what was actually done — which post was commented on, what the email said — because the sequence is only useful on the second pass if it records that.

A step can also be **blocked**: the comment steps need a LinkedIn activity to comment on, and the email step needs an email address. `blocked_reason` is a property on the step rather than a template filter, so the board and the person page get the same answer without either registering anything.

SECTION 14

Where the data stands

Read from `data/peopleintel.db` at build time. This is the sample Apollo export plus whatever research has been run against it.

27 CONTACTS	21 COMPANIES	26 IDENTITY COMPLETE	21 RESEARCHED
----------------	-----------------	-------------------------	------------------

47

LINKEDIN ACTIVITIES

35

WEB FINDINGS

37

INTEREST CHIPS

11

FOCUS LINES

What the numbers say

- **Both status combinations are present.** 20 contacts are identity-complete and researched, 6 are complete but not researched yet, and 1 researched while still missing an identity field — the state a single combined status could not express (section 6).
- **Every stored web finding is corroborated:** 35 of 35. That is not a coincidence — anything that cleared none of the three signals was never stored (section 9).
- **16 contacts have at least one web finding** and 15 have at least one LinkedIn activity, out of 27. The gap is the uneven coverage described in section 9, not a failure.
- **44 of 47 activities were found by search**, 3 pasted by hand. Types: 22 post, 14 tagged, 11 comment.
- **4 contacts show a designation on the web**, which is where the drift flags and the /reports stale list come from. 1 shows a different posting profile than the file records.
- **11 focus lines and 37 interest chips** across 27 contacts — the rest returned nothing rather than guessing, which is the intended behaviour when evidence is thin.
- **3 comment drafts** exist. The rest of the activity rows failed the substance check, and say so (section 10).
- **9 outreach steps** have been created. Steps are created one at a time as the sequence progresses, so this counts real progress rather than a plan.

SECTION 15

Build history

8 commits, oldest last. Each one is a behaviour change rather than a checkpoint — several of them are described in detail in the sections above.

Commit	Date	What changed
3f98985	2026-09-05	Keep the LinkedIn panel current, and the drafts ready to paste Added the standalone Refresh LinkedIn button with its own in-flight flag, and the recommended comment drafts with the substance check in front of them.
ed97ccc	2026-09-02	Move the company profile above the opportunities panel on Home The company panel argues for the pitch below it, so it has to be read first.
f29a4ae	2026-09-02	Attribute LinkedIn activity by evidence, not just the CSV slug Rules 2 and 3 of section 8, plus <code>linkedin_observed</code> and the drift warning. Reverted the “unverified” badge experiment.
19898c1	2026-09-01	Segment the list by country and category Both filters derived on read — section 12.
6b62692	2026-09-01	Merge pull request #1 from Tanveenkaur13/seema
47d31d5	2026-09-01	make some changes
640c07b	2026-09-01	Ignore personal Claude Code settings override
feba0b3	2026-09-01	PeopleIntel: lead research dashboard with outreach sequencing The initial build: the nine steps, the six tables, the two statuses, the importer's tolerances, corroborated web research, and the outreach sequence.

A pattern worth naming

Three separate times, a looser rule was built, measured against the real data, found to be wrong, and removed: name-token matching for LinkedIn activity, the “identity unconfirmed” badge on web findings, and the merged-blob search in the opportunities matcher. In each case the replacement stores less and says more about what it does store. The measurements are quoted in sections 8, 9 and 11 rather than summarised away, because they are the argument for the current rule.

SECTION 16

Limits and what is not built

Stated here rather than discovered later.

Not built	Why, and what it would take
Email generation	Deliberately out of scope. The schema has room for it, and step 4 of the outreach sequence already tracks the email a person sends.
Direct phone numbers	The Apollo export has none for any of the 25 contacts — only company switchboards. Direct lines need a paid waterfall enrichment, so the field is shown as an optional gap and never holds a contact at Needs Enrichment.
Email deliverability	<code>resolve.py</code> claims only that an address is <i>published as theirs</i> — which is why what it writes lands in <code>email_status</code> as “found on the web” rather than “valid”. Proving a mailbox exists needs a verification provider and another key.
Person photos	Not in the CSV, and LinkedIn’s images cannot be hotlinked or reused. The avatar is initials; company logos come from favicons.
Authentication	The header shows a fixed user. Real accounts are not implemented, so this is a tool for one team on a trusted network.
Refresh scheduling	Research is fetched once. The LinkedIn panel can now be refreshed on demand, but nothing re-runs on a schedule — so a “2 days ago” activity will read “9 days ago” a week later, and web findings drift out of date.
Serverless deployment	SQLite needs a persistent disk. On Vercel and similar the filesystem is ephemeral and writes vanish between requests. Point <code>app/db.py</code> at Neon or Supabase and nothing else changes.

And the limits that are features

- A contact whose posts a search engine has never indexed gets an empty LinkedIn panel, and it is not padded.
- A contact with thin evidence gets no interest chips and no focus line.
- A relevant page that never repeats the person’s title and employer in its snippet is dropped along with the namesakes.
- A capability with nothing to point at is left off the opportunities panel.
- Each of those is a deliberate trade of recall for precision, made because the output is read by a person who is about to contact a stranger — and a wrong row costs more there than a missing one.